

ASSIGNMENT-10.3

2303A51296

Batch-10

Task-1

Prompt: [actasai-assistedbugdetection](#).

[#writeafactorial of a numcode and test it with\(5\)and identify thelocigalbugin the codeandalsoexplainwhythatbugoccurs\(eg:off-by-oneerror\).](#)

code:

```
deffactorial(n):
```

```
    result = 1
```

```
    foriinrange(1,n):
```

```
        result *= i
```

```
    return result
```

```
print(factorial(5))
```

TestOutput: Thefunctionreturns24insteadoftheexpected120.

LogicalBugIdentified:

Thisisanoff-by-oneerror.Theloopusesrange(1,n)whichgeneratesnumbersfrom1ton-1.

Whythebugoccurs:

range(1,n)withn=5produces:[1,2,3,4]

Thecalculationbecomes: $1*1*2*3*4=24$ (whichis $4!$)

Thefinalnumber5isnevermultiplied,sowegetonenumbershort.

#correctversion:

```
deffactorial(n):
```

```
    result = 1
```

```
    foriinrange(1,n+1):
```

```
        result *= i
```

```
    returnresult
```

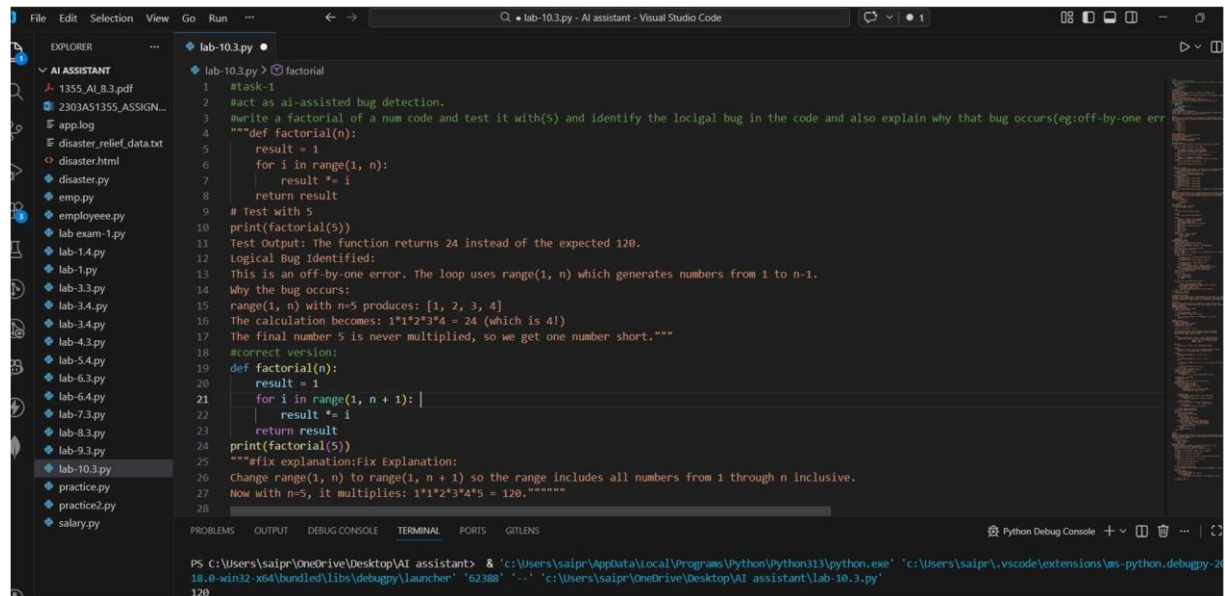
```
print(factorial(5))
```

FixExplanation:

Changerange(1,n) torange(1,n +1)so therange includesall numbers from1through n inclusive.

Nowwithn=5,itmultiplies:1*1*2*3*4*5=120.

Output:



```
1 #task-1
2 #act as ai-assisted bug detection.
3 #write a factorial of a num code and test it with(5) and identify the locigal bug in the code and also explain why that bug occurs(eg:off-by-one err
4
5 ***def factorial(n):
6     result = 1
7     for i in range(1, n):
8         result *= i
9     return result
10
11 # Test with 5
12 print(factorial(5))
13 Test Output: The function returns 24 instead of the expected 120.
14 Logical Bug Identified:
15 This is an off-by-one error. The loop uses range(1, n) which generates numbers from 1 to n-1.
16 Why the bug occurs:
17 range(1, n) with n=5 produces: [1, 2, 3, 4]
18 The calculation becomes: 1*1*2*3*4 = 24 (which is 4!)
19 The final number 5 is never multiplied, so we get one number short.***
20
21 #correct version:
22 def factorial(n):
23     result = 1
24     for i in range(1, n + 1):
25         result *= i
26     return result
27
28 print(factorial(5))
29
30 ***#fix explanation:Fix Explanation:
31 Change range(1, n) to range(1, n + 1) so the range includes all numbers from 1 through n inclusive.
32 Now with n=5, it multiplies: 1*1*2*3*4*5 = 120.*****
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
```

CodeAnalysis:

- ❑ PurposeoftheFunction – Thefactorial(n) function calculates thefactorial ofa numbert,n,whichistheproductofallpositiveintegersfrom1 ton(e.g.,5!= 1×2×3×4×5 = 120).
- ❑ LogicalBugIdentified– Theoriginalcodeusedrange(1,n),whichexcludesthe valuen.Becauseofthis,thelooponlymultipliesnumbersfrom1 ton-1.
- ❑ TypeofError –Thisis anoff-by-oneerror, acommonprogrammingmistake wherealooprunsonesteplessormorethanrequired.
- ❑ EffectoftheBug– Whentestingwithn=5,theloopmultiplies1×2×3×4,giving 24insteadof120,whichactuallycomputes4!insteadof5!.
- ❑ FixApplied– Changingthelooptorange(1,n+1)includesthenumbereininthe multiplication,ensuringthefactorialiscalculatedcorrectly.

Task-2

Prompt:Analyze the given Python function for readability, parameter naming issues, and lack of documentation, then rewrite it with descriptive names, a completed docstring, input validation, and exception handling for division by zero.

#Also compare the original and improved versions and test them with valid inputs, division by zero, and invalid operations.

Code:

```
def calc(a,b,op): if
```

```
    op == '+':
```

```
        return a+b
```

```
    elif op == '-':
```

```
        return a-b
```

```
    elif op == '*':
```

```
        return a * b
```

```
    elif op == '/':
```

```
        return a/b
```

refactored:

```
def perform_arithmetic_operation(first_number,second_number,operation):
```

```
    if not isinstance(first_number, (int,float)) or not isinstance(second_number,(int, float)):
```

```
        raise TypeError("Both operands must be numbers")
```

```
    if operation == '/' and second_number == 0:
```

```
        raise ValueError("Cannot divide by zero") if
```

```
    operation == '+':
```

```
        return first_number + second_number
```

```
    elif operation == '-':
```

```
        return first_number - second_number
```

```
    elif operation == '*':
```

```
        return first_number * second_number
```

```
    elif operation == '/':
```

```
        return first_number/second_number
```

else:

```
    raise ValueError("Invalid operation")
```

Taking input

```
a=float(input("Enterfirstnumber:"))
```

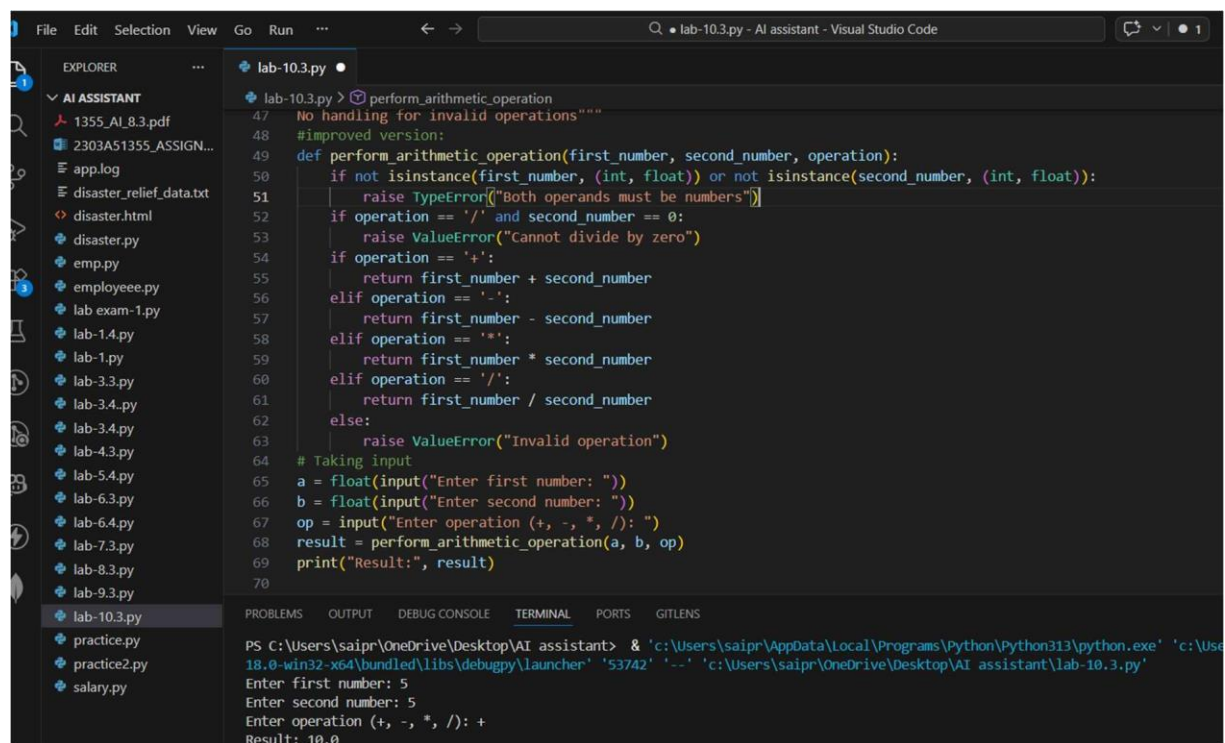
```
b = float(input("Enter second number: "))
```

```
op=input("Enteroperation(+,-,*,/):")
```

```
result=perform_arithmetic_operation(a, b,op)
```

```
print("Result:", result)
```

Output:

The screenshot shows the Visual Studio Code interface. The Explorer pane on the left lists files including 'lab-10.3.py'. The main editor displays the Python code for 'lab-10.3.py', which includes a function 'perform_arithmetic_operation' with type checking and error handling, followed by input prompts and a call to the function. The TERMINAL pane at the bottom shows the command prompt execution: 'PS C:\Users\saipr\OneDrive\Desktop\AI assistant> & 'c:\Users\saipr\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\saipr\OneDrive\Desktop\AI assistant\lab-10.3.py'', followed by user input '5', '5', and '+', resulting in 'Result: 10.0'.

CodeAnalysis:

- ❑ The improved code uses clear parameter names (first_number, second_number,operation)whichimprovesreadability.
- ❑ Itincludesinputvalidationtoensuretheoperandsarenumbers.

- ❑ It handles division by zero using an exception to prevent errors.
- ❑ It checks for invalid operations and raises an error if the operation is not supported.
- ❑ The program also accepts user input, allowing the function to be tested easily.

Task-3

Prompt: Analyze the given Python function for PEP8 violations such as naming conventions, spacing, indentation, and formatting.

Refactor the code to make it PEP8 compliant while preserving its functionality and verify it with sample inputs.

Also briefly explain how AI-based automated code reviews can help streamline code reviews in large development teams.

Code:

```
def Checkprime(n):
```

```
    for i in range(2,n):
```

```
        if n % i == 0:
```

```
            return False
```

```
    return True
```

refactorized:

```
def check_prime(number: int) ->
```

```
bool:
```

```
    """
```

```
    Check if a number is prime. """
```

```
    if not isinstance(number, int):
```

```
        raise TypeError("Input must
```

```
be an
```

```
integer") if nu
```

```
mber < 2:
```

```

        raise ValueError("Number
must be >= 2")
    if number == 2:
        return True
    if number % 2 == 0:
        return False
    divisor = 3
    while divisor * divisor <=
number:
        if number % divisor == 0:
            return False
        divisor += 2
    return True
#Taking user input
num = int(input("Enter a number:
"))
if check_prime(num):
    print(num, "is a Prime number")
else:
    print(num, "is not a Prime
number")

```

Output:

```

lab-10.3.py X
lab-10.3.py > check_prime
82 def check_prime(number: int) -> bool:
83     """
84     Check if a number is prime.
85     """
86     if not isinstance(number, int):
87         raise TypeError("Input must be an integer")
88     if number < 2:
89         raise ValueError("Number must be >= 2")
90     if number == 2:
91         return True
92     if number % 2 == 0:
93         return False
94     divisor = 3
95     while divisor * divisor <= number:
96         if number % divisor == 0:
97             return False
98         divisor += 2
99     return True
100 # Taking user input
101 num = int(input("Enter a number: "))
102 if check_prime(num):
103     print(num, "is a Prime number")
104 else:
105     print(num, "is not a Prime number")
106 ---
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GITLENS
PS C:\Users\sai\r\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\r\AppData\Local\Programs\Python\Python313\python
18.0-win32-x64\bundled\libs\debugpy\launcher' '62924' '--' 'c:\Users\sai\r\OneDrive\Desktop\AI assistant\lab-10.3
Enter a number: 6
6 is not a Prime number

```

CodeAnalysis:

- ❑ The function name was changed from Checkprime to check_prime to follow PEP8 snake_case naming convention.
- ❑ The refactored code has proper spacing and indentation, improving readability.
- ❑ Input validation ensures the value is an integer and greater than or equal to 2.
- ❑ The algorithm checks divisibility up to the square root of the number, making it more efficient.
- ❑ AI-based code review helped detect style issues and bugs quickly, improving code quality in large teams.

Task-4

Prompt: [#Review the given Python function for readability, naming, and edge cases.](#)

[#Improve it with better naming, type hints, input validation, and a configurable multiplier, then refactor the code and briefly reflect on AI's role in code reviews.](#)

Code:

from typing import List, Union

```
def double_even_numbers(numbers:List[Union[int, float]]) ->
```

```
List[Union[int, float]]:
```

```
    ifnotisinstance(numbers,list):
```

```
        raise TypeError("Input must be a list")
```

```
    return [
```

```
        num*2
```

```
        fornuminnumbers
```

```
        ifisinstance(num,(int,float))andnum%2==0
```

```
    ]
```

```
#refactorized:
```

```
fromtypingimportList,Union
```

```
def double_even_integers(
```

```
    numbers: List[Union[int, float]],
```

```
    include_floats: bool = False
```

```
)->List[Union[int,float]]: """
```

```
    Doubleallevenintegersinalist. """
```

```
    ifnotisinstance(numbers,list):
```

```
        raiseTypeError(f"Inputmustbealist,got
```

```
{type(numbers).name}") if
```

```
    len(numbers) == 0:
```

```
        return []
```

```
    validated_numbers=[]
```

```
    for idx, num in enumerate(numbers):
```

```
        ifnotisinstance(num,(int,float)):
```

```
            raiseTypeError(
```

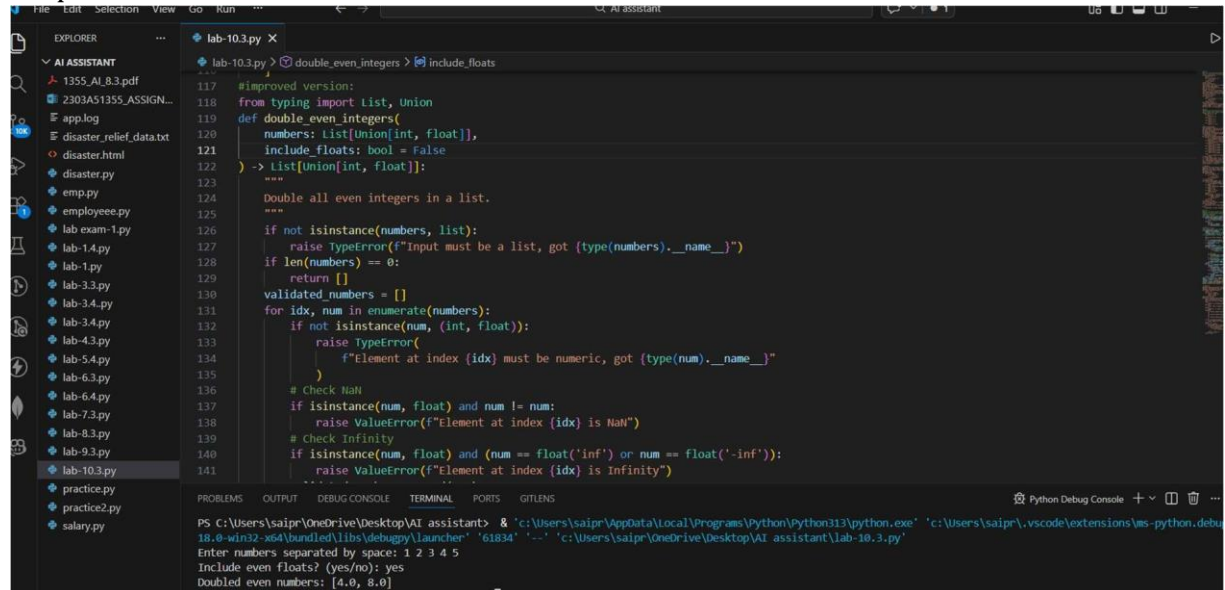
```

        f"Element at index {idx} must be numeric, got
{type(num).name}"
    )
    # Check NaN
    if isinstance(num, float) and num != num:
        raise ValueError(f"Element at index {idx} is NaN") #
    # Check Infinity
    if isinstance(num, float) and (num == float('inf') or num ==
float('-inf')):
        raise ValueError(f"Element at index {idx} is Infinity")
    validated_numbers.append(num)
result = []
for num in validated_numbers:
    is_even = False
    if isinstance(num, int):
        is_even = num % 2 == 0
    elif isinstance(num, float) and include_floats:
        if num == int(num):
            is_even = int(num) % 2 == 0
    if is_even:
        result.append(num * 2)
return result

nums = list(map(float, input("Enter numbers separated by space:
").split()))
choice = input("Include even floats? (yes/no): ").lower()
include = True if choice == "yes" else False
result = double_even_integers(nums, include)
print("Doubled even numbers:", result)

```

Output:



```
lab-10.3.py
117 #improved version:
118 from typing import List, Union
119 def double_even_integers(
120     numbers: List[Union[int, float]],
121     include_floats: bool = False
122 ) -> List[Union[int, float]]:
123     """
124     Double all even integers in a list.
125     """
126     if not isinstance(numbers, list):
127         raise TypeError(f"Input must be a list, got {type(numbers).__name__}")
128     if len(numbers) == 0:
129         return []
130     validated_numbers = []
131     for idx, num in enumerate(numbers):
132         if not isinstance(num, (int, float)):
133             raise TypeError(
134                 f"Element at index {idx} must be numeric, got {type(num).__name__}"
135             )
136         # Check NaN
137         if isinstance(num, float) and num != num:
138             raise ValueError(f"Element at index {idx} is NaN")
139         # Check Infinity
140         if isinstance(num, float) and (num == float('inf') or num == float('-inf')):
141             raise ValueError(f"Element at index {idx} is Infinity")
142     # ... (rest of the function code) ...
```

PS C:\Users\sai\r\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\r\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sai\r\.vscode\extensions\ms-python.debug-18.0-win32-x64\bundle\libs\debugpy\launcher' '61834' '-' 'c:\Users\sai\r\OneDrive\Desktop\AI assistant\lab-10.3.py'

Enter numbers separated by space: 1 2 3 4 5
Include even floats? (yes/no): yes
Doubled even numbers: [4.0, 8.0]

CodeAnalysis:

- ❑ BetterNaming:Function named `double_even_integers` and variables are clearer and improve readability.
- ❑ TypeHints: Uses `List[Union[int,float]]` to clearly define input and output types.
- ❑ InputValidation: Checks if input is a list and ensures elements are numeric.
- ❑ EdgeCases: Handles empty lists, NaN, and Infinity values.
- ❑ AI inCodeReviews: AI helps detect errors, improve code quality, and speed up code reviews.

Task-5

Prompt: [Analyze the Python function for time complexity and performance issues when processing large datasets.](#)

[#Suggest improvements and provide an optimized version \(using built-in functions or NumPy if suitable\), then compare execution time before and after and briefly discuss readability vs performance trade-offs.](#)

Code:

```
defsum_of_squares(numbers):
```

```
    total = 0
```

```
    for num in numbers:
```

```
        total += num**2
```

```
    return total
```

refactorized:

```
from typing import List, Union
```

```
def sum_of_squares_basic(numbers: List[Union[int, float]]) -> float: """
```

```
    Basic corrected version - indentation fix only.
```

```
    Calculate the sum of squares of all numbers in a list. """
```

```
    if not isinstance(numbers, list):
```

```
        raise TypeError(f"Input must be a list, got {type(numbers).name}") total = 0
```

```
    for num in numbers:
```

```
        if not isinstance(num, (int, float)):
```

```
            raise TypeError(
```

```
                f"All elements must be numeric, got {type(num).name}"
```

```
            )
```

```
            total += num**2
```

```
    return total
```

```
nums = list(map(float, input("Enter numbers separated by space:
```

```
").split())) result = sum_of_squares_basic(nums)
```

```
print("Sum of squares:", result)
```

Output:

```
163 for num in numbers:
164     total += num ** 2
165     return total"""
166 #improved version:
167 from typing import List, Union
168 def sum_of_squares_basic(numbers: List[Union[int, float]]) -> float:
169     """
170     Basic corrected version - indentation fix only,
171     calculate the sum of squares of all numbers in a list.
172     """
173     if not isinstance(numbers, list):
174         raise TypeError(f"Input must be a list, got {type(numbers).__name__}")
175     total = 0
176     for num in numbers:
177         if not isinstance(num, (int, float)):
178             raise TypeError(
179                 f"All elements must be numeric, got {type(num).__name__}"
180             )
181         total += num ** 2
182     return total
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS GIT LENS

PS C:\Users\sai\r\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\r\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sai\r\.vscode\extensions\ms-python.18.0-win32-x64\bundled\libs\debugpy\launcher' '63636' '--' 'c:\Users\sai\r\OneDrive\Desktop\AI assistant\lab-10.3.py'

Enter numbers separated by space: 1 2 3 4 5

Sum of squares: 55.0

CodeAnalysis:

- ❑ TimeComplexity: The function has $O(n)$ time complexity because it processes each element once.
- ❑ Performance: For large datasets, Python loops may be slower. Built-in functions or NumPy can improve speed.
- ❑ InputValidation: The improved code checks if input is a list and elements are numeric.
- ❑ CodeQuality: Type hints and error handling improve readability and maintainability.
- ❑ Trade-off: Simple loops are easy to read, while optimized methods give better performance.